

Capitolo 47 — PROT-018 — Internal Async Dependency Resolution

Pubblico	Lettori tecnici e non tecnici del WCM Process & Memory Book
Versione	FROZEN-47
Data master	2026-09-02
Stato	FROZEN
Source path	wcm/documentation/process-memory-book/ chapters/47_prot_018_internal_async_dependency_resolution.md
Source SHA	fd2a33b
Release	2026-09-04T09:44:50.631Z

GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.

PARTE VII — Il Libro dei Protocolli WCM Stato: FROZEN **Data:** 2026-09-02 **Scope:** WCM generale, domain-agnostic

47.0 Aspettare non significa essere bloccati

Un'organizzazione può fermarsi per due ragioni molto diverse.

La prima è un vero ostacolo: manca una decisione, manca un'autorità, manca una capacità indispensabile oppure si è verificato un errore che impedisce di proseguire in sicurezza.

La seconda è più sottile: una parte del sistema sta semplicemente aspettando che un'altra parte completi un'attività già nota, autorizzata e risolvibile all'interno dello stesso sistema.

Confondere queste due situazioni produce un effetto pericoloso. Un'attesa tecnica interna viene trattata come un problema da portare a una persona, come un blocco di progetto o addirittura come motivo per interrompere il meccanismo che avrebbe dovuto riprendere il lavoro.

PROT-018 — Internal Async Dependency Resolution nasce per evitare proprio questo errore.

Il suo principio fondamentale è semplice:

Se una dipendenza nasce e può essere risolta interamente dentro il WCM, la sua attesa non è, da sola, un gate umano e non è, da sola, un blocker di progetto.

Il protocollo non elimina l'attesa. Le assegna il significato corretto e costruisce il modo per riprendere il lavoro senza duplicarlo.

47.1 Che cos'è una dipendenza interna asincrona

Una dipendenza è qualcosa che deve accadere prima che un'altra attività possa proseguire.

È interna quando il risultato può essere prodotto da un componente, servizio o routine che appartiene al perimetro operativo del WCM e non richiede, per quella specifica esecuzione, una nuova decisione esterna.

È asincrona quando il risultato non arriva necessariamente nello stesso istante in cui viene richiesto.

Un esempio astratto aiuta.

Immaginiamo un processo che, prima di completare una fase, debba ottenere una verifica aggiornata da un servizio interno. Il processo principale ha già fatto tutto ciò che poteva fare. La verifica è stata richiesta correttamente, ma richiede qualche tempo per essere prodotta.

In quel momento il processo non dovrebbe:

- chiedere a una persona di decidere qualcosa che non c'è da decidere;
- ripetere il lavoro già completato;
- dichiararsi definitivamente bloccato;
- fingere che la verifica sia già pronta;
- spegnere il meccanismo che dovrà riprenderlo.

Dovrebbe invece registrare in modo durevole:

“Sto aspettando questa precisa dipendenza interna. Quando sarà pronta, riprenderò da questo preciso punto.”

Questo è il problema che PROT-018 formalizza.

47.2 Il rischio del falso blocker

Un sistema cognitivo può osservare che manca qualcosa e concludere rapidamente: “non posso continuare”.

Ma nel WCM questa frase non è sufficiente.

Prima bisogna capire **che cosa manca, chi può produrlo e se l'attesa richiede davvero una decisione.**

La differenza può essere rappresentata così:

```
MANCA UN RISULTATO
  ↓
È UNA DECISIONE / AUTHORITY UMANA?
├ SÌ → vero gate umano
└ NO
  ↓
È UNA DIPENDENZA INTERNA GIÀ RISOLVIBILE?
├ SÌ → WAITING_INTERNAL_DEPENDENCY
└ NO → classificare il problema reale
```

Il falso blocker nasce quando il secondo caso viene trattato come il primo o come un'impossibilità definitiva.

Questo può produrre stalli inutili, richieste di intervento umano prive di valore e, soprattutto, perdita di continuità operativa.

47.3 L'oggetto persistente: ricordare cosa stiamo aspettando

L'attesa deve sopravvivere alla singola sessione.

Per questo PROT-018 non si affida a una frase nella conversazione o alla memoria temporanea dell'agente. La dipendenza viene rappresentata come un oggetto persistente del runtime.

La baseline corrente prevede oggetti sotto un'area dedicata alle dipendenze interne del progetto. Il formato tecnico non è importante per comprendere il concetto. Importano invece le informazioni che devono restare ricostruibili:

- identità stabile della dipendenza;
- progetto a cui appartiene;
- tipo di risultato richiesto;
- stato corrente della dipendenza;
- punto del workflow dopo il quale il risultato deve essere considerato fresco;
- workflow che consumerà quel risultato;
- transizione esatta da cui riprendere;
- assenza di effetti automatici su authority e scheduler.

In termini umani, il sistema deve poter rispondere a cinque domande:

1. **cosa sto aspettando?**
2. **per quale workflow?**
3. **rispetto a quale stato deve essere valido il risultato?**
4. **quando sarà pronto, da dove devo riprendere?**
5. **questa attesa cambia authority o scheduler?**

Se queste risposte non sono persistite, l'attesa rischia di trasformarsi in perdita di contesto.

47.4 Il ciclo PENDING → READY → CONSUMED

Il cuore del protocollo è un piccolo ciclo di vita.

```
PENDING
↓
READY
↓
CONSUMED
```

Questi tre stati hanno significati molto diversi.

PENDING

La dipendenza è nota e registrata, ma il risultato richiesto non è ancora pronto.

PENDING non significa “qualcuno deve decidere”. Significa:

| *“Il sistema sa cosa manca e sa quale componente interno deve produrlo.”*

READY

Il risultato è stato prodotto e possiede l'evidenza necessaria per essere usato.

READY non significa ancora che il workflow abbia già utilizzato quel risultato. Significa soltanto che la dipendenza è disponibile per il consumer.

CONSUMED

Il workflow ha verificato il risultato, lo ha utilizzato nel punto previsto e non deve ripetere la stessa dipendenza.

Questo ultimo stato è essenziale per l'idempotenza: se il sistema riparte una seconda volta, vede che quel lavoro è già stato consumato e non lo rigenera artificialmente.

47.5 PENDING non è WAITING_AUTHORITY

Questa distinzione è una delle più importanti dell'intero protocollo.

Un gate umano esiste quando serve una decisione o un'autorità che il sistema non possiede.

Una dipendenza interna **PENDING** esiste invece quando la decisione è già stata presa e il sistema sta aspettando un risultato tecnico interno necessario per continuare.

`WAITING_AUTHORITY`

→ manca un diritto di decidere / procedere

`WAITING_INTERNAL_DEPENDENCY`

→ l'autorità è sufficiente, manca un risultato interno già richiesto

Confondere i due stati significa trasferire agli esseri umani problemi che l'organizzazione dovrebbe risolvere da sola.

PROT-018 vieta quindi che un semplice **PENDING** venga trasformato automaticamente in richiesta di authority.

47.6 PENDING non è un motivo per rifare il lavoro

Quando una run termina mentre una dipendenza è ancora **PENDING**, una run successiva potrebbe essere tentata di ricominciare la fase che l'ha generata.

È proprio ciò che il protocollo vuole evitare.

Se il workflow ha già completato alcuni passaggi e ha registrato la dipendenza, quei passaggi restano completati.

La regola è:

```
DIPENDENZA PENDING
→ conserva il checkpoint
→ conserva la next transition
→ non ripetere completed steps
→ attendi la risoluzione interna
```

Questo collega PROT-018 direttamente al principio di Resume Priority visto nei capitoli precedenti.

La continuità non consiste nel “riprovare tutto”. Consiste nel riprendere dal punto esatto in cui il workflow era rimasto.

47.7 Chi risolve la dipendenza

La baseline prevede un dispatcher deterministico per le dipendenze interne.

Il termine può sembrare tecnico, ma il concetto è semplice: esiste un componente incaricato di osservare una richiesta interna strutturata, eseguire il servizio previsto e aggiornare lo stato della dipendenza.

Il flusso è:

```
WORKFLOW
→ registra dipendenza PENDING
→ dispatcher interno la rileva
→ esegue il servizio richiesto
→ verifica il risultato
→ dipendenza READY
```

Il punto importante è che il workflow cognitivo non deve restare attivo a “guardare” l'attesa e non deve improvvisare un nuovo percorso.

La dipendenza ha un proprietario operativo chiaro e un ciclo persistente.

Questo riduce la necessità di usare ragionamento probabilistico per una meccanica che può essere espressa con stati e regole precise.

47.8 Essere READY non basta: serve la prova che il risultato sia quello giusto

Uno dei problemi più insidiosi delle attività asincrone è utilizzare un risultato formalmente recente ma costruito sullo stato sbagliato.

Immaginiamo che il workflow richieda una verifica **dopo** una modifica importante. Se il servizio restituisce una verifica prodotta prima di quella modifica, il dato può essere perfettamente valido nel proprio contesto e comunque non essere valido per la transizione corrente.

Per questo PROT-018 non considera sufficiente la sola data o ora.

La baseline richiede una verifica di **lineage/freshness** rispetto a un punto preciso della storia del sistema.

In linguaggio semplice:

Non chiediamo soltanto “quanto è recente questo risultato?”, ma “questo risultato comprende davvero lo stato che doveva comprendere?”.

È una differenza fondamentale.

Un orologio può dirci che qualcosa è nuovo. La lineage può dirci se discende dal punto corretto della storia.

47.9 Il significato di `required_after`

Per rendere verificabile il concetto precedente, la dipendenza può registrare un riferimento che significa:

“Il risultato che aspetto deve essere prodotto a partire da uno stato non precedente a questo punto.”

Nella baseline tecnica questo riferimento è rappresentato da `required_after_sha`.

Non è necessario conoscere Git o un hash per comprendere il principio. Possiamo immaginarlo come un timbro verificabile applicato alla versione minima della realtà che il risultato deve conoscere.

Se il servizio restituisce un risultato che discende da quel punto, la freshness causale può essere verificata.

Se non lo fa, il risultato non dovrebbe diventare `READY` soltanto perché ha un timestamp più nuovo.

47.10 `READY` non decide automaticamente l'esito semantico

Un'altra distinzione importante: una dipendenza `READY` prova che il risultato richiesto è disponibile e correttamente collegato al punto necessario del workflow.

Non prova automaticamente che quel risultato sia favorevole.

Per esempio, un servizio interno può produrre una verifica valida che segnala una condizione degradata. Il consumer deve ancora interpretare, secondo il processo e i protocolli applicabili, se quella condizione sia rilevante per la transizione corrente.

```
READY
→ EVIDENZA DISPONIBILE E FRESCA

READY
≠
ESITO SEMANTICO AUTOMATICAMENTE POSITIVO
```

Questa separazione protegge il confine tra meccanica deterministica e valutazione cognitiva.

Il dispatcher può garantire che il dato sia pronto. Non deve inventare il significato operativo che spetta al workflow consumer quando quel significato non è determinabile meccanicamente.

47.11 Il consumer: leggere prima di dichiarare un blocker

Quando il workflow riparte, il worker deve leggere le dipendenze dichiarate **prima** di classificare un nuovo blocker.

La logica canonica è:

```
PENDING
→ non duplicare lavoro
→ conserva next transition
→ attendi

READY
→ verifica evidence
→ consuma alla resume transition
→ continua il workflow

CONSUMED
→ non ripetere la dipendenza

FAILED
→ classifica il failure reale
```

Questa regola impedisce un errore frequente: una nuova sessione vede che manca ancora un output nel workflow, ignora che esista già una dipendenza READY e dichiara il problema da capo.

La Persistent Organizational Memory deve servire proprio a evitare questa amnesia operativa.

47.12 FAILED: quando l'attesa diventa davvero un problema

Il protocollo ammette anche lo stato **FAILED**, ma lo riserva a un fallimento interno verificato che non sia semplicemente un'attesa normale.

Anche in questo caso, però, **FAILED** non significa automaticamente “capability gap”.

Prima di concludere che il sistema non possiede la capacità necessaria, si applicano le regole di verifica della capability previste da PROT-011.

La distinzione è:

```
SERVIZIO NON HA ANCORA FINITO
→ PENDING

SERVIZIO HA FALLITO IN MODO VERIFICATO
→ FAILED

CAPABILITY DAVVERO ASSENTE?
→ verificarlo con evidence corrente
```

Questa graduazione impedisce che un incidente tecnico temporaneo venga trasformato in una conclusione strutturale falsa.

47.13 Scheduler ownership: una dipendenza non può spegnere chi deve riprenderla

Uno dei principi più pratici di PROT-018 riguarda il meccanismo di esecuzione periodica.

La dipendenza interna non possiede authority sullo scheduler.

Nella baseline, l'effetto sullo scheduler è esplicitamente **NONE**.

Il motivo è logico: se una dipendenza PENDING portasse il worker a disabilitare il meccanismo che dovrebbe tornare a controllarla, il sistema potrebbe costruire da solo il proprio deadlock.

```
PENDING
→ ASPETTA
→ MA NON SPEGNERE IL MECCANISMO DI RIPRESA
```

Lo stesso vale per un failure interno: il fatto che qualcosa sia andato storto non conferisce automaticamente al componente che lo osserva l'autorità di modificare lo scheduler.

Authority operativa e stato della dipendenza restano separati.

47.14 Un esempio quotidiano: il documento che aspetta una verifica interna

Immaginiamo una procedura amministrativa.

Un documento è stato preparato e prima della fase successiva deve ricevere una verifica automatica di coerenza da un ufficio interno. La verifica non è istantanea.

La procedura corretta non è telefonare subito al direttore per chiedergli “cosa facciamo?”, perché non manca una decisione del direttore.

Non è nemmeno riscrivere il documento ogni dieci minuti.

Il comportamento corretto è:

1. registrare che la verifica è stata richiesta;
2. identificare esattamente quale versione del documento deve essere verificata;
3. attendere che l'ufficio interno completi il controllo;
4. verificare che l'esito riguardi proprio quella versione o una sua evoluzione valida;
5. usare l'esito una sola volta;
6. proseguire dal passaggio successivo già previsto.

Se invece l'ufficio dichiara che il controllo non può essere eseguito per un problema reale, allora il problema viene classificato per ciò che è, senza trasformare retroattivamente tutta l'attesa precedente in un blocker.

47.15 Il flusso completo

La baseline di PROT-018 può essere sintetizzata così:

```
WORKFLOW ATTIVO
↓
SERVE UN RISULTATO INTERNO NON IMMEDIATO
↓
PERSISTI DIPENDENZA PENDING
↓
CONSERVA WORKFLOW + NEXT TRANSITION
↓
DISPATCHER DETERMINISTICO
↓
SERVIZIO INTERNO
↓
VERIFICA LINEAGE / FRESHNESS
↓
READY
↓
WORKER SUCCESSIVO LEGGE LA DIPENDENZA
↓
VERIFICA EVIDENCE
↓
CONSUMED
↓
RIPRENDE DALLA RESUME TRANSITION
↓
PROT-009 FINO ALLA VERA STOP CONDITION
```

Questa sequenza separa tre responsabilità:

- il workflow sa **che cosa sta aspettando**;
- il dispatcher sa **come ottenere il risultato interno**;
- il consumer sa **come usare quel risultato nel contesto del workflow**.

47.16 Relazione con Resume Priority

PROT-018 non sostituisce PROT-009. Lo completa in un caso specifico.

PROT-009 dice che un workflow incompleto deve essere ripreso dalla sua `next_transition`, senza rieseguire i passaggi già completati.

PROT-018 aggiunge:

se la next transition dipende da un risultato interno asincrono, quella dipendenza deve essere persistita, risolta e consumata senza trasformare l'attesa in un nuovo workflow o in un falso gate.

Possiamo quindi leggere i due protocolli insieme:

```
PROT-009
→ DOVE DEVO RIPRENDERE?
```

```
PROT-018
→ COSA FACCIO SE PER RIPRENDERE ASPETTO UN SERVIZIO INTERNO?
```

47.17 Relazione con PROT-011

PROT-011 protegge il WCM da un'altra conclusione affrettata: dichiarare che una capability non esiste senza averlo verificato nel runtime corrente.

PROT-018 usa questo principio quando una dipendenza interna fallisce.

Un **FAILED** verificato può significare molte cose:

- errore temporaneo;
- problema del servizio;
- permesso mancante;
- dipendenza malformata;
- capability realmente assente.

Soltanto l'ultima categoria giustifica un vero **CAPABILITY_GAP**, e anche quella deve essere verificata.

La lezione è coerente con l'intero WCM: **non trasformare una mancanza osservata in una spiegazione più ampia di quanto l'evidenza consenta.**

47.18 Deterministico e cognitivo: chi decide cosa

PROT-018 mostra molto bene la separazione tra Deterministic Core e Cognitive Core.

Sono buoni candidati per gestione deterministica:

- identità della dipendenza;
- lifecycle PENDING / READY / CONSUMED;
- trigger del dispatcher;
- verifica strutturale dei campi;
- controllo della lineage;
- persistenza dello stato;
- riconoscimento che **scheduler_effect=NONE**;
- prevenzione di consumo duplicato.

Può invece richiedere valutazione cognitiva il significato del risultato prodotto.

Per esempio, una verifica interna può essere tecnicamente valida e segnalare una condizione **DEGRADED**. Stabilire se quella condizione blocchi davvero la transizione corrente può dipendere dal significato e dal contesto.

La macchina può stabilire con precisione **che il risultato è quello giusto**. Non sempre può stabilire da sola **che cosa quel risultato significhi per la decisione**.

47.19 Failure mode principali

Il protocollo protegge da alcuni errori ricorrenti.

PENDING TRATTATO COME HUMAN GATE

→ falso bisogno di authority

PENDING TRATTATO COME BLOCKER

→ falso stop di progetto

PENDING → RESTART DEL LAVORO

→ duplicazione

READY SENZA LINEAGE

→ rischio di usare evidence stale

READY = PASS SEMANTICO AUTOMATICO

→ confusione tra meccanica e significato

FAILED = CAPABILITY_GAP AUTOMATICO

→ conclusione non verificata

DIPENDENZA MODIFICA LO SCHEDULER

→ rischio di deadlock auto-prodotto

CONSUMED IGNORATO

→ lavoro interno ripetuto

La maggior parte di questi failure mode nasce dallo stesso errore: **non distinguere lo stato tecnico della dipendenza dallo stato semantico e di authority del workflow.**

47.20 Cosa produce PROT-018

L'output del protocollo non è semplicemente “il servizio interno ha finito”.

È una dipendenza risolta in modo tracciabile e consumabile, tale che il workflow possa riprendere senza perdere continuità.

Il risultato desiderato comprende:

- dipendenza identificata;
- stato persistente;
- punto minimo di freshness verificabile;
- consumer e resume transition noti;
- evidence READY verificata;
- consumo registrato;
- nessuna duplicazione dei passaggi precedenti;
- nessuna authority inventata;
- nessun effetto improprio sullo scheduler.

Il vero successo è quindi la **ripresa corretta del workflow**, non la sola chiusura del task interno.

47.21 Maturity: cosa possiamo dire e cosa no

PROT-018 è una baseline **ACTIVE / FIELD-VALIDATED** del WCM corrente.

La sua introduzione deriva da una failure reale osservata durante l'esecuzione di un workflow: un controllo interno asincrono necessario non era stato rappresentato come dipendenza persistente e questo aveva prodotto uno stallo ripetibile. La correzione ha introdotto un oggetto di dipendenza esplicito, un dispatcher deterministico e una verifica di freshness basata sulla lineage.

Nel libro manteniamo volutamente astratto il caso di origine: ciò che interessa è la lezione metodologica, non il progetto nel quale è emersa.

La field validation dimostra che il pattern è stato provato nel contesto WCM che lo ha generato. Non dimostra che ogni tipo di dipendenza asincrona, ogni infrastruttura o ogni dominio sia già coperto universalmente.

La formulazione corretta è quindi:

- il protocollo è parte della baseline corrente;
- il lifecycle e le invarianti principali hanno evidenza operativa;
- la generalizzazione a ulteriori tipi di dipendenza richiede applicazione e verifica coerenti;
- non ogni failure asincrona è automaticamente risolvibile internamente.

47.22 La regola da ricordare

Se dovessimo conservare una sola idea di questo capitolo, sarebbe questa:

Quando il WCM sa già cosa sta aspettando, chi deve produrlo e da dove riprendere, l'attesa interna non deve diventare amnesia, blocker o richiesta umana: deve diventare stato persistente, risoluzione verificata e ripresa.

Il capitolo precedente proteggeva il momento in cui il sistema modifica qualcosa di persistente. Questo capitolo protegge un altro confine delicato: il tempo che passa tra una richiesta interna e il momento in cui il suo risultato diventa disponibile.

L'affidabilità non richiede che tutto accada nello stesso istante. Richiede che, anche quando qualcosa accade dopo, il sistema sappia ancora **che cosa stava facendo, che cosa aspettava e da dove continuare.**

Source Map essenziale

Fonte primaria:

- [wcm/process-book/protocols/PROT-018_INTERNAL_ASYNC_DEPENDENCY_RESOLUTION.md](#) — ACTIVE / FIELD-VALIDATED.

Fonti collegate necessarie:

- [wcm/process-book/protocols/PROT-009_CONTIGUOUS_WORKFLOW_EXECUTION.md](#) — Resume Priority, checkpoint e true stop condition;
- [wcm/process-book/protocols/PROT-011_CAPABILITY_EVIDENCE_CHECK_BEFORE_BLOCK.md](#) — verifica della capability prima di `CAPABILITY_GAP`;

- `WCM_AGENT_START.md` — bootstrap runtime-first e priorità delle dipendenze dichiarate dal workflow.

Maturity qualifier: baseline WCM attiva con field validation nel contesto che ha originato il protocollo; nessun claim di validazione universale cross-domain.